

COSC 3P93 Assignment 2

Due date: December 7th, 2025 at 23:59 (11:59pm)

Delivery method: the student needs to delivery the assignment only through Sakai.

Delivery contents: document with answers and [Java, C, C++] codes if applicable (see [Submission instructions](#)).

Attention: check the [Late Assignment Policy](#).

Performance [32]

Question 1.1 Task Latency and Task Throughput. [8]

A fintech application uses a multi-threaded system with four worker threads (T_1, T_2, T_3, T_4) to process a queue of independent financial analysis tasks (Tasks A through J). Each task involves calculating a volatility score for a stock trade packet. Tasks are processed immediately by the first available thread.

The performance monitoring system records the execution details for a batch of 10 tasks. Table 1 provides the thread assignment, the exact moment the thread began processing the task (Start Time), and the duration of the task's computation (Task Duration).

Assumptions: Thread scheduling overhead is negligible, and all threads begin their work search at Time $t = 0$. Task Latency is equivalent to Task Duration, as there is no waiting time once execution begins.

Table 1: Thread Execution Log for 10 Tasks

Task ID	Assigned Thread	Start Time (s)	Task Duration (s)	Finish Time (s)
A	T1	0.0	2.5	2.5
B	T2	0.0	3.0	3.0
C	T3	0.0	1.0	1.0
D	T4	0.0	2.0	2.0
E	T3	1.0	4.0	5.0
F	T4	2.0	1.5	3.5
G	T1	2.5	4.5	7.0
H	T2	3.0	2.0	5.0
I	T4	3.5	3.0	6.5
J	T3	5.0	2.0	7.0

Answer the following questions:

- (i) Define **Task Latency** and determine the **Average Task Latency** for the entire batch of 10 tasks.
- (ii) Define **Task Throughput** and determine the **Overall Task Throughput** in the observed time interval.

Question 1.2 Performance analysis. [16]

Given the following three parallel algorithms (names), implement them in any language (C/C++/Java) that suits you (it is ok to find sources online provided that you cite them). Provide their implementation code attached to the assignment submission. Also, provide a performance analysis (**Speedup**, **Efficiency**, and **Cost**) by collecting their run times on 1, 2, and 4 cores (at least up to 2 cores in case a 4-core machine is not accessible).

- (a) Parallel Bubble Sort
- (b) Parallel String Search in multiple text files
- (c) Parallel Fibonacci

Question 1.3 Workflow analysis. [8]

Given the following algorithms, provide the following for each of them: (i) workflow graph (DAG); and (ii) workflow analysis (**Workspan**, **Lower Bound**, and **Upper bound** (Brent's Lemma)). *Both bounds relate to a given system size (number of processors), so explicitly and clearly determine a size for your analysis and calculate them.*

(a) Matrix Multiplication -> you can assume the dimension of the matrices as 4x4

```
1 Input: matrices A and B
2 Let C be a new matrix of the appropriate size
3 For i from 1 to n:
4   For j from 1 to p:
5     Let sum = 0
6     For k from 1 to m:
7       Set sum <- sum + Aik x Bkj
8     Set Cij <- sum
9 Return C
```

where the load is divided in slices to assign to threads/processes. Each slice corresponds to 1 or more consecutive i rows in A (1st loop in the pseudocode).

(b) Parallel Fibonacci -> You can assume a call P-Fib(4)

```
1 P-Fib(n) {
2   if n $\leq$ 1
3     return n;
4   else {
5     x = spawn P-Fib(n-1); /*a thread is spawned to calculate this branch */
6     y = P-Fib(n-2);
7     sync /* parents waits for child thread (join) */
8     return x + y;
9   }
10 }
```

Parallel Design Patterns [20]

Question 2.1 Dependencies. [5]

You are analyzing five independent code snippets executed concurrently by two threads, **Thread1**(T_1) and **Thread2**(T_2). Both threads access the same shared memory locations: an array **A**, scalar variables **X** and **Y**, and a matrix **M**.

A **data dependency** arises when the final result of a program is determined by the specific order of execution of core instructions accessing the same memory location.

(i) **Identify the Dependency Type:** Identify the specific type of dependency present between the instructions (I_n , where $n \in \mathbb{N}$ the increasing sequence defines the ordering of the instructions) as one of the following:

- **True (Flow) Dependency**
- **Anti-Dependency**
- **Output Dependency**
- **No Dependency**

(ii) **Explain the Conflict:** Briefly explain the conflict by referencing the memory location(s) and the specific operations (Read/Write) performed by each instruction.

Table 2: Concurrent Instruction Pairs for Dependency Analysis

Excerpt	Thread 1 (T_1) Snippet	Thread 2 (T_2) Snippet
1	$I_1 : P = P + 1$ $I_3 : A[10] = X + 10$ $I_5 : Q = Q - 1$	$I_2 : O = O + 2$ $I_4 : Y = A[10] + 5$ $I_6 : R = R - 2$
2	$I_1 : P = P + 1$ $I_3 : X = A[5]$ $I_5 : R = R - 1$	$I_2 : P = P + 2$ $I_4 : A[5] = 2 \times Y$ $I_6 : O = O - 2$
3	$I_1 : P = P + 1$ $I_3 : M[0][1] = 1$ $I_5 : Q = Q - 1$	$I_2 : T = T + 2$ $I_4 : M[0][1] = 2$ $I_6 : S = S - 2$
4	$I_1 : P = P + 1$ $I_3 : X = 15$ $I_5 : L = L - 1$	$I_2 : W = W + 2$ $I_4 : Y = 20$ $I_6 : Q = Q - 2$
5	$I_1 : S = S + 1$ $I_3 : A[8] = Y$ $I_5 : U = U - 1$	$I_2 : P = P + 2$ $I_4 : Y = A[8] + 1$ $I_6 : Q = Q - 2$

Question 2.2 Loop-carried dependency. [5]

You are tasked with optimizing three computationally intensive loop structures for a multi-core system using OpenMP. Analyzing dependencies is the first step in determining if and how a loop can be safely parallelized.

For each of the following C/C++ pseudocode excerpts in Table 3, assume A,B, and C are large, shared global arrays, and N is the loop size.

For each excerpt, analyze the loop to answer the following:

(i) Identify Dependency: Does a **loop-carried dependency** exist? If so, state the dependency type (forward or backward) and the specific array index relationship.

(ii) Explain Coupling: Explain the nature of the dependency.

(iii) Parallelism Opportunity: Can the loop be safely parallelized using an OpenMP for directive (#pragma omp for)? If yes, explain how. If no, explain why.

Table 3: Loop Excerpts for Dependency Analysis

Excerpt	C/C++ Pseudocode
1	<pre>1 for (int i = 1; i < N; i++) { 2 A[i] = A[i-1] * 0.5 + B[i]; 3 }</pre>
2	<pre>1 for (int i = 10; i < N; i++) { 2 A[i] = A[i-10] + C[i] * X; 3 }</pre>
3	<pre>1 for (int i = 1; i < N; i++) { 2 A[i] = 2.0 * A[i] + C[i]; 3 B[i-1] = A[i] * 0.5; 4 }</pre>

Question 2.3 Parallel Control Patterns. [5]

Parallel Control Patterns define how computation is managed, synchronized, and orchestrated across multiple processes. These patterns are fundamental to structuring parallel algorithms.

Analyze the four independent OpenMP code excerpts below in Tables 4 and 5. Assume **A**, **B** are large shared arrays, **N** is the loop size, and **P** is the total number of threads.

(i) Identify the Pattern: For each excerpt, identify the primary parallel control pattern demonstrated from the following list: Fork-Join, Map, Stencil, and Reduction.

(ii) Explain the Mechanism: Briefly explain how the identified pattern controls the execution flow and data dependence in the given OpenMP context, focusing on the explicit OpenMP directives and synchronization points.

Table 4: MPI Excerpts for Parallel Control Pattern Analysis 1/2

Excerpt	C/C++ MPI Pseudocode
1	<pre>1 #pragma omp parallel for 2 for (int i = 0; i < N; i++) { 3 B[i] = sin(A[i]) + 1.0; 4 } 5 // Threads operate independently on contiguous sections of data.</pre>
2	<pre>1 // Assume A is a 1D array where A[0] and A[N-1] are boundary conditions 2 #pragma omp parallel for shared(A, B) 3 for (int i = 1; i < N - 1; i++) { 4 // Computation requires neighbor data 5 B[i] = (A[i-1] + A[i] + A[i+1]) / 3.0; 6 } 7 // Requires implicit synchronization after this loop to ensure B is complete</pre>

Table 5: MPI Excerpts for Parallel Control Pattern Analysis 2/2

Excerpt	C/C++ MPI Pseudocode
3	<pre> 1 \begin{lstlisting} 2 // threads are spawned here (Outer call) 3 #pragma omp parallel 4 { 5 // Outer loop distributes 'num_tasks' among the initial 6 // thread team 7 #pragma omp for schedule(static) 8 for (int i = 0; i < num_tasks; i++) { 9 10 // Inner parallel region: each thread in the outer 11 // team 12 // becomes a master, potentially spawning a new 13 // thread team 14 #pragma omp parallel 15 { 16 perform_task(i); 17 } // Inner call 18 } 19 } // Outer call 20 printf("All tasks complete.\n"); </pre>
4	<pre> 1 double result = 0.0; 2 3 #pragma omp parallel for 4 for (int i = 0; i < N; i++) { 5 #pragma omp atomic update 6 result += A[i]; 7 } 8 printf("Result: %f\n", result); </pre>

Question 2.4 Parallel Data Management Patterns. [5]

In parallel computing, specific algorithmic structures called **Parallel Data Management Patterns** are used to organize the flow of data between processing stages to maximize performance. Recognizing these patterns is crucial for efficient parallel programming.

Analyze the five independent OpenMP excerpts in Tables 6 and 7. Assume **A**, **B**, **C**, **D** are large shared arrays, **N** is the loop size, and **T** is a constant.

(i) Identify the Pattern: For each excerpt, identify the primary parallel data management pattern demonstrated from the following list: Pack, Pipeline, Gather, and Scatter.

(ii) Explain the Mechanism: Briefly explain how the identified pattern works in the given code, focusing on the movement or organization of data relative to the threads.

Table 6: OpenMP Excerpts for Data Management Pattern Analysis 1/2

Excerpt	C/C++ Pseudocode
1	<pre> 1 MPI_Request reqs[local_n]; 2 // Each process iterates its local block, potentially sending results elsewhere 3 for (int i = 0; i < local_n; i++) { 4 int global_index = C[i]; 5 int dest_rank = global_index / local_n; 6 7 // Calculate the result (A[i]*2) 8 double result = local_a[i] * 2.0; 9 10 // Non-blocking send the result to the owner of D[global_index] 11 MPI_Isend(&result, 1, MPI_DOUBLE, dest_rank, global_index, 12 MPI_COMM_WORLD, &reqs[i]); 13 } 14 // Requires synchronization: MPI_Waitall(...) </pre>
2	<pre> 1 // Assumes sequential rank execution: rank 0 -> rank 1 -> ... 2 if (rank > 0) { 3 // Receive input from predecessor (rank - 1) 4 MPI_Irecv(B_in, local_n, MPI_DOUBLE, rank - 1, TAG_FILTER, 5 MPI_COMM_WORLD, &recv_req); 6 } 7 8 // Compute Stage 1 locally (e.g., B[i] = A[i] * 0.9) 9 10 if (rank < size - 1) { 11 // Send output to successor (rank + 1) 12 MPI_Isend(B_out, local_n, MPI_DOUBLE, rank + 1, TAG_FILTER, 13 MPI_COMM_WORLD, &send_req); 14 } 15 // Wait for required input before Stage 2 16 if (rank > 0) MPI_Wait(&recv_req, MPI_STATUS_IGNORE); 17 18 // Stage 2 computation on received data 19 for (int i = 0; i < local_n; i++) { 20 C[i] = B_in[i] + 10; 21 } </pre>

Table 7: OpenMP Excerpts for Data Management Pattern Analysis 2/2

Excerpt	C/C++ Pseudocode
3	<pre> 1 // Buffer to hold the incoming data from other processes 2 double D_temp[local_n]; 3 MPI_Request reqs[local_n]; 4 int count = 0; 5 6 for (int i = 0; i < local_n; i++) { 7 int global_index = C[i]; // C[i] contains the global indexes 8 int source_rank = global_index / local_n; // translates to source 9 10 // Request (or setup receive) data from owner process: MPI_Irecv 11 // Often done via MPI_Alltoallv or a dedicated request mechanism 12 13 // For simplicity, models the *need* for external data 14 // Assuming D_temp[i] now holds D[global_index] 15 // ... 16 A[i] = D_temp[i] * 2.0; //operation with retrieved data 17 } 18 // Communication complexity (MPI_Alltoallv or point-to-point requests) is hidden </pre>
4	<pre> 1 // 1. Local Filtering 2 int local_count = 0; 3 for (int i = 0; i < local_n; i++) { 4 if (local_a[i] > T) { 5 local_b[local_count++] = local_a[i]; 6 } 7 } 8 // 2. Determine Global Offsets 9 int total_count; 10 int displacement; 11 MPI_Scan(&local_count, &displacement, 1, MPI_INT, MPI_SUM, 12 MPI_COMM_WORLD); 13 // Displacement now holds the starting index for this process's 14 // data in the global array 15 displacement -= local_count; 16 // 3. Assemble Global Array 17 // MPI_Gatherv used to assemble local_b into final global B array at 18 // root </pre>

OpenMPI [12]

Question 3.1 OpenMPI communication. [6]

The following C/C++ code excerpt uses MPI to perform a 1D Stencil computation (a 3-point average) on a large array **A**, where each process needs to exchange boundary data with its neighbours.

Analyze the code and the explicit MPI function calls used.

```
1  int rank, size;
2  int left_neighbor, right_neighbor;
3  double left_send, right_send;
4  double left_recv, right_recv;
5
6  MPI_Init(NULL, NULL);
7  MPI_Comm_rank(MPI_COMM_WORLD, &rank);
8  MPI_Comm_size(MPI_COMM_WORLD, &size);
9
10 left_neighbor = (rank == 0) ? MPI_PROC_NULL : rank - 1;
11 right_neighbor = (rank == size - 1) ? MPI_PROC_NULL : rank + 1;
12
13 left_send = local_array[1];
14 right_send = local_array[local_N];
15
16 MPI_Sendrecv(&left_send, 1, MPI_DOUBLE, left_neighbor, 0,
17             &right_recv, 1, MPI_DOUBLE, right_neighbor, 0,
18             MPI_COMM_WORLD, MPI_STATUS_IGNORE);
19
20 MPI_Sendrecv(&right_send, 1, MPI_DOUBLE, right_neighbor, 1,
21             &left_recv, 1, MPI_DOUBLE, left_neighbor, 1,
22             MPI_COMM_WORLD, MPI_STATUS_IGNORE);
23
24 local_array[0] = left_recv;
25 local_array[local_N + 1] = right_recv;
26
27 MPI_Barrier(MPI_COMM_WORLD);
28
29 // Computation phase starts here...
30
31 MPI_Finalize();
```

Question 3.2 OpenMPI collective communication. [6]

The following C/C++ code excerpt uses MPI collective communication routines to initialize a parallel K-Means clustering algorithm. The goal is to distribute data, select initial centroids, calculate the total variance, and synchronize results across all processes.

Analyze the code and the explicit MPI collective function calls used.

Code Excerpt: K-Means Initialization Routine

```
1  int rank, size;
2  int K = 4; // Number of clusters
3  double local_sum;
4  double global_sum_variance;
5  double max_variance;
6  double *local_data = NULL; // Pointer to local subset of data
7  double *centroids;        // K centroids (same size on all processes)
8
9  MPI_Init(NULL, NULL);
10 MPI_Comm_rank(MPI_COMM_WORLD, &rank);
11 MPI_Comm_size(MPI_COMM_WORLD, &size);
12
13 if (rank == 0) {
14     local_data = (double*)malloc(local_N * sizeof(double));
15 }
16 MPI_Scatter(global_data, local_N, MPI_DOUBLE,
17             local_data, local_N, MPI_DOUBLE,
18             0, MPI_COMM_WORLD);
19
20 centroids = (double*)malloc(K * sizeof(double));
21 MPI_Bcast(centroids, K, MPI_DOUBLE, 0, MPI_COMM_WORLD);
22
23 local_sum = calculate_local_variance(local_data, local_N);
24
25 MPI_Reduce(&local_sum, &global_sum_variance, 1, MPI_DOUBLE,
26            MPI_SUM, 0, MPI_COMM_WORLD);
27
28 double *all_local_means = NULL;
29 if (rank == 0) {
30     all_local_means = (double*)malloc(size * sizeof(double));
31 }
32 MPI_Gather(&local_sum, 1, MPI_DOUBLE,
33            all_local_means, 1, MPI_DOUBLE,
34            0, MPI_COMM_WORLD);
35
36 MPI_Allreduce(&global_sum_variance, &max_variance, 1, MPI_DOUBLE,
37               MPI_MAX, MPI_COMM_WORLD);
38
39 double *all_local_sums_to_all = (double*)malloc(size * sizeof(double));
40 MPI_Allgather(&local_sum, 1, MPI_DOUBLE,
41               all_local_sums_to_all, 1, MPI_DOUBLE,
42               MPI_COMM_WORLD);
43
44 MPI_Finalize();
```

Parallel Algorithms [20]

Question 4.1 Parallel algorithm implementation. [20]

Please refer to the enclosed parallel algorithms. Assume we receive these codes without any documentation and comments. They work well, as defined in textbook (lecture notes – slides). Please address the following questions for each algorithm.

- a) Parallel Quicksort using CUDA. (you can try to compile+run it in your machine or for free in Google Colab)
 - (i) Compile the code and provide the compilation call. Also, does the code compile? If yes, what is the output of the program? If no, what is missing?
 - (ii) Explain the algorithm through the code. Use the lecture notes about parallel quicksort and identify in the code (show the excerpts) where those major points of the pseudocode are happening.
 - (iii) Does the entire code run in the GPU? Explain your answer.
 - (iv) What does `CUDA_CHECK()` do in the code? Explain its role and how it works.
 - (v) What is `SEQUENTIAL_CUTOFF` in the code? Explain.

Tools [8]

Question 5.1 Factors of performance. [8]

- (i) What are the factors of sequential performance? (ii) What are the factors of parallel performance?

Parallel Program Design [8]

Question 6.1 Parallel design methodology. [8]

- (i) Explain the methodological design of parallel programs. (ii) List and explain each of its parts/phases.

Marking Scheme

Marks will be awarded for completeness and demonstration of understanding of the material. It is crucial that students fully show their knowledge when providing solutions concisely. Quality and conciseness of solutions are considered when awarding marks. Every code added to the originals should be well commented on and explicitly indicated in the Java files; lack of clarity may lead students to lose marks. Thus, students are recommended to keep it simple and clear.

Submission Material

The submission for this assignment will consist of a single part:

1. The **Answer document (PDF)**. The answers to the assignment questions should all be included in this document. Also, include the description of the source Java/C/C++ code, especially the compilation and execution instructions. There is no need to worry about the formatting of this document; students must follow the provided Latex template.
 - **Latex template - a must for writing your assignment.** For writing the description file, use the Latex template enclosed in this assignment (update it accordingly!). Students do not need to install Latex software on their computers. Students can write it through Overleaf on their browser (it is a free tool). Just upload the latex template to an Overleaf project; it should compile/render the tex file gracefully.
2. The **C/C++/Java code respective to the assignment questions**. The code must be documented (commented) properly. The code should compile and run properly. The compilation and execution of students' codes should not rely on any IDE.
 - **Compilation.** Provide the command for compiling the source code from the command line.
 - **Running.** Provide the command for running the compiled code from the command line.

Submission

Submission is to be a PDF (description document), and text code files. All submissions should be performed electronically through D2L (BrightSpace).

Students must guarantee that their code is legible, clear, and succinct. Remember that any questionable implementation decisions or copying from any source might negatively impact marks. If students still have questions about which file types are acceptable, please inquire prior to submission. Note that it is not the markers' fault if they cannot mark an project due to submitting an unreasonable or uncommon file format.

The submission consists of two files and must submitted through D2L:

- **Description Document in a PDF file.**
- **Code in a Zip file.**

Late Assignment Policy

There is no late submission policy. No late submission is allowed.

Plagiarism

Students are expected respect academic integrity and deliver evaluation materials that are only produced by themselves. Any copy of content, text or code, from other students, books, web, or any other source is not tolerated. Similarity checking tools (Turnitin) will be used on submitted materials. If there is any indication that an activity contains any part copied from any source, a case will be open and brought to a plagiarism committee's attention. In case plagiarism is determined, the activity will be cancelled, and the author(s) will be subject to the university regulations.

For further information on this sensitive subject, please refer to the document below:

<https://brocku.ca/academic-integrity/>